Módulo II

Comunicaciones seguras

- Tema 3: Protección de la confidencialidad
- Tema 4: Cifrados de clave secreta
- Tema 5: Distribución de claves

Gestión de claves en criptografía simétrica

- Generación, distribución, almacenamiento, tiempo de vida, destrucción y aplicación de las claves de acuerdo con una política de seguridad.
- Longitud de la clave
- Elección correcta de la clave para evitar ataque de diccionario. Generación seudoaleatoria.
- Tiempo de vida de las claves: Fecha de caducidad porque cuanto más tiempo se use una clave:
  - aumenta la probabilidad de que se comprometa
  - mayor será el daño si la clave se compromete, ya que toda la información protegida con esa clave queda al descubierto.
  - mayor será la tentación de alguien para intentar desbaratarla.
  - en general es más fácil realizar criptoanálisis con mucho texto cifrado con la misma clave.





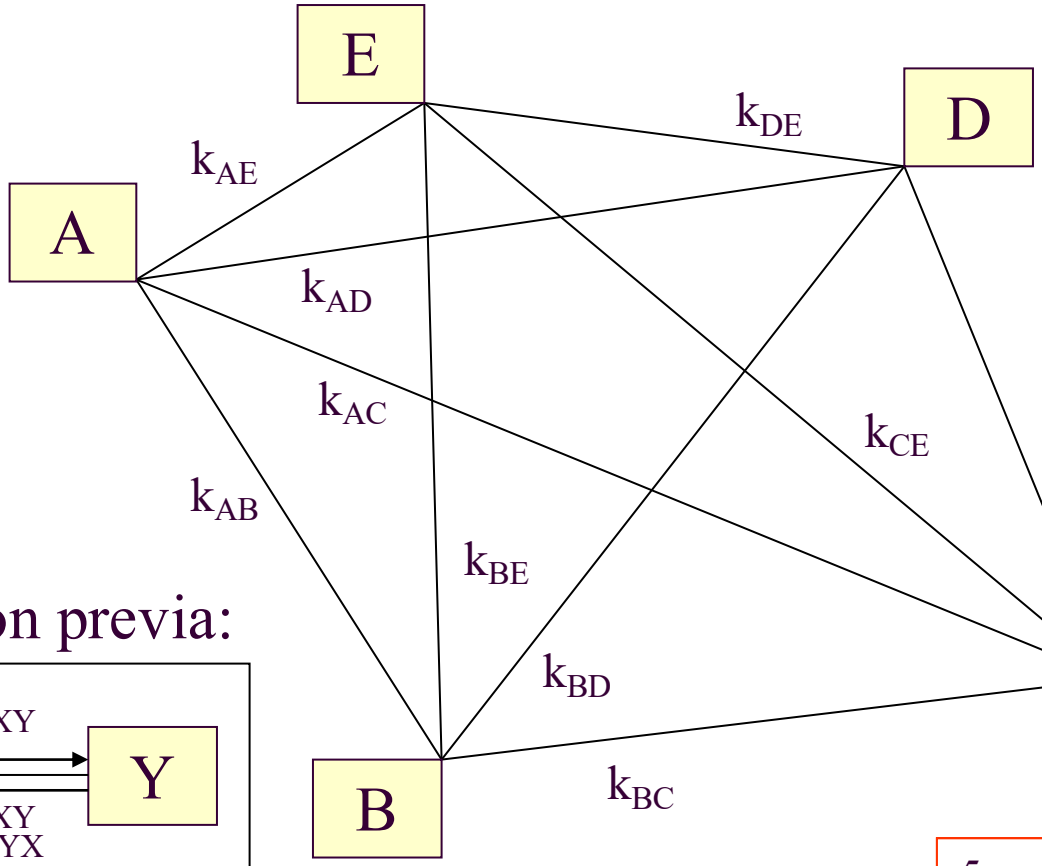
Gestión de Claves

- Jerarquía del espacio de claves:
  - Claves de sesión: empleadas para cifrar los datos de usuario y que son actualizadas frecuentemente.
  - Claves de cifrado de claves: para proteger las claves de sesión.
  - Claves maestras: distribuidas manualmente y se utilizan para proteger las de cifrado de claves cuando son intercambiadas.





Distribución de Claves Secretas

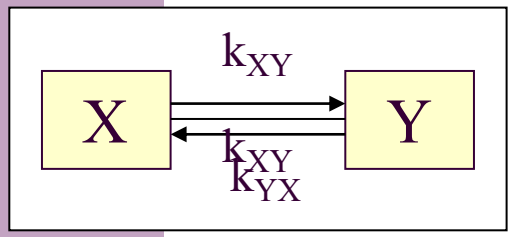


N = nº de claves

Número Claves:

$$\frac{n\,(n-1)}{2}$$

Definición previa:



5 usuarios: N = 10

La gestión de las claves es una tarea muy difícil

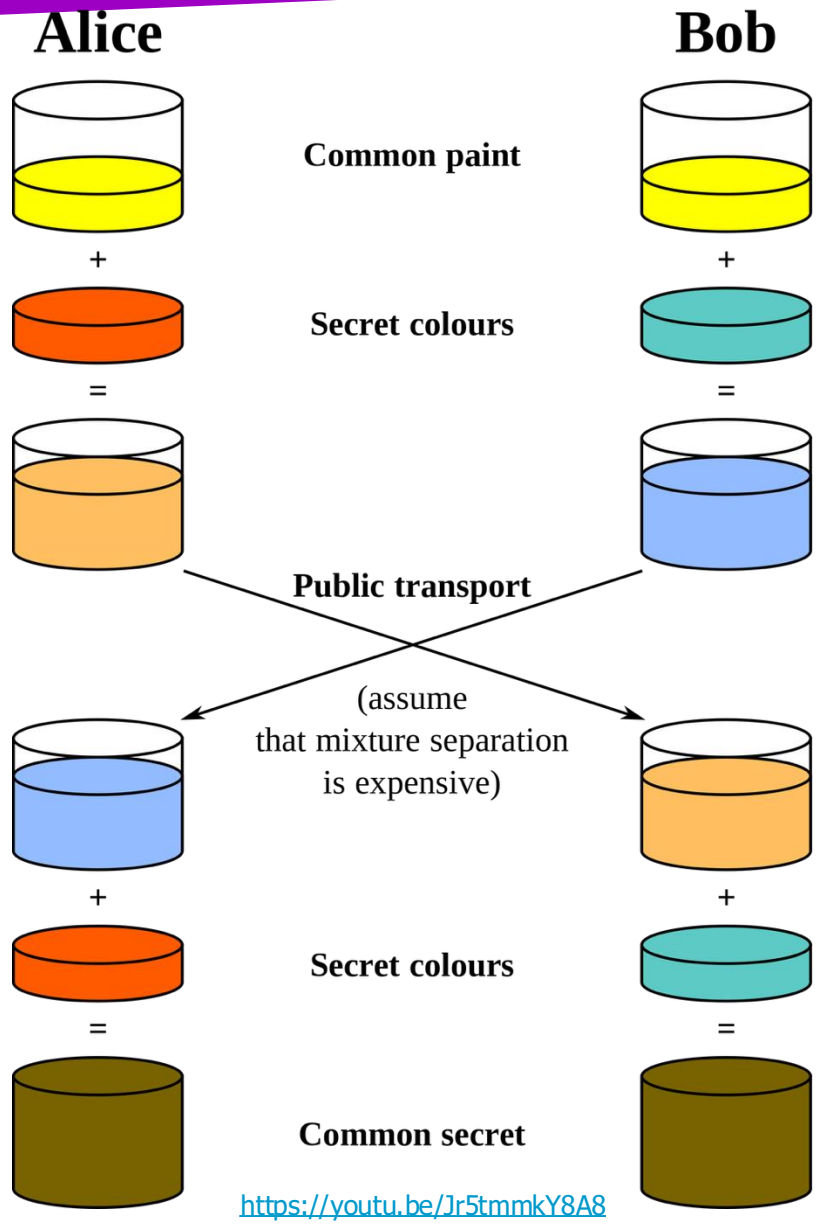
Nacimiento de la Clave Pública

1976
(MIT)



o 1970?
(Oficina criptoanalítica británica)
James H. Ellis





https://youtu.be/Jr5tmmkY8A8

Logaritmo Discreto

PLD: Problema del **Logaritmo Discreto**:

Dados y, g y p, encontrar x tal que $g^x=y \pmod p$

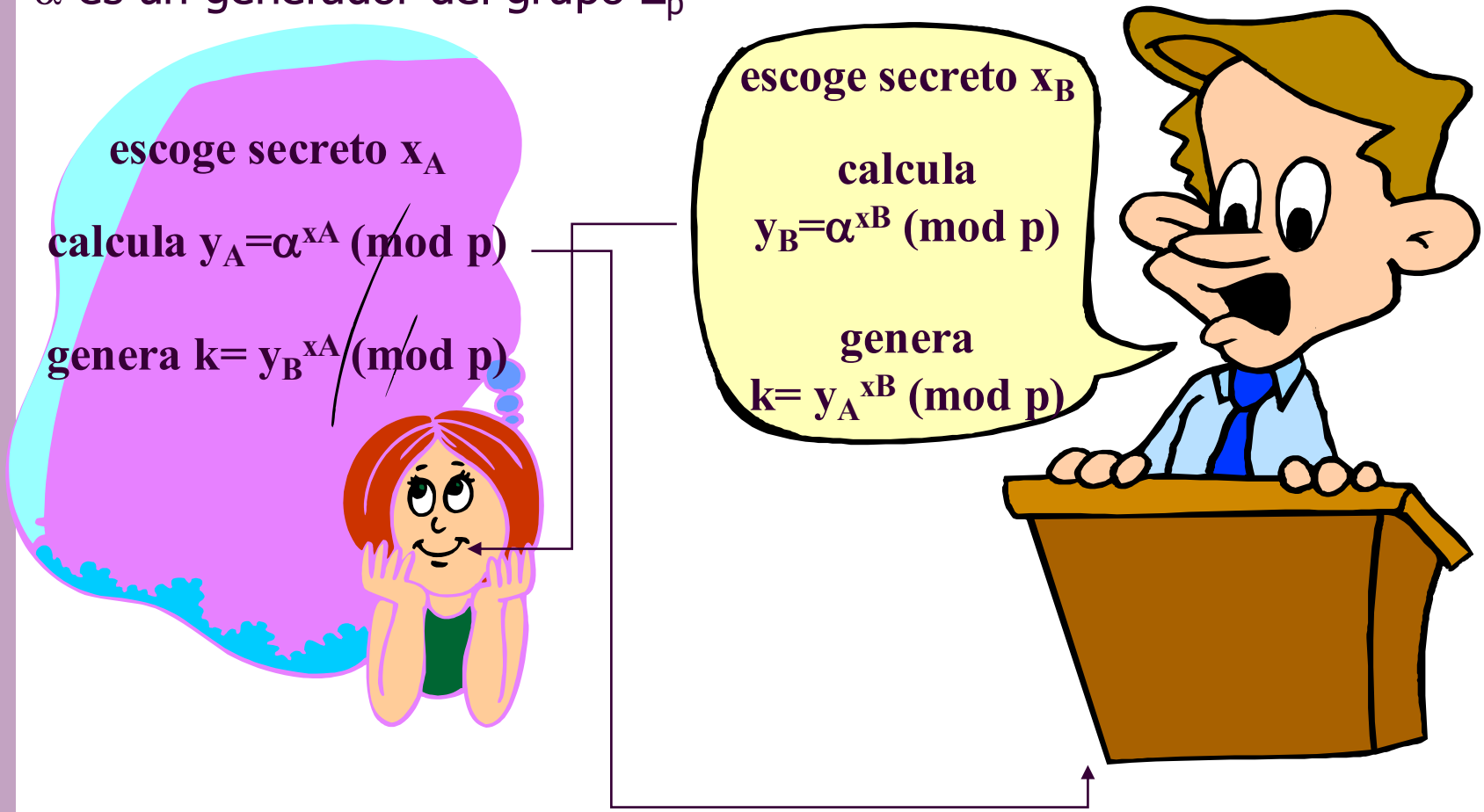
- –Ejemplos:
  - –Si $13=3^x \pmod{17} \Rightarrow x=4$.
  - –Si $7=3^x \pmod{13} \Rightarrow$ No hay x
- –Si p es primo, el PLD tiene complejidad equivalente al problema de la factorización.

Hay métodos para resolverlo en algunas circunstancias.

Algoritmo de Diffie-Hellman

Protocolo de establecimiento de clave secreta compartida entre dos partes que no han tenido contacto previo, utilizando un canal inseguro, y de manera anónima.

α es un generador del grupo Z_p



Algoritmo de Diffie-Hellman

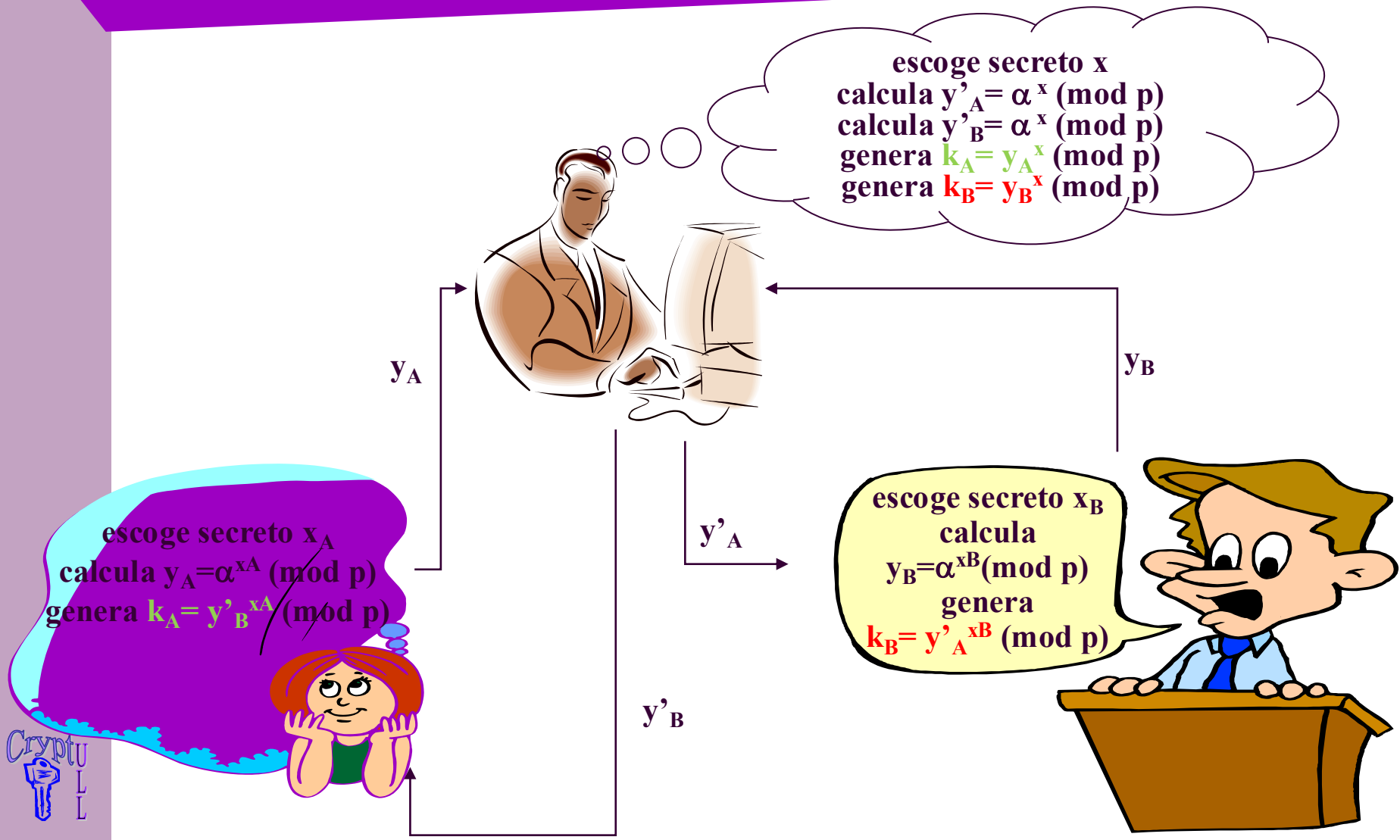
Ejemplo:

A	$p = 53,\ \alpha = 2$	B
$x_A = 29$		$x_B = 19$
$y_A = 2^{29} \equiv 45 \pmod{53}$	y_A →	$y_B = 2^{19} \equiv 12 \pmod{53}$
	← y_B	$y_A = 45$
$y_B = 12$		
$K = 12^{29} \equiv 21 \pmod{53}$		$K = 45^{19} \equiv 21 \pmod{53}$

El protocolo DH se usa en:

- **TLS/SSL** (navegación segura por Internet)
- **Whatsapp** (mensajería por móvil)
- **OpenSSL** (software libre de cifrado y firma)
- **Ipsec** (protocolo de bajo nivel para paquetes IP)
- **TOR** (red para proteger anonimato)

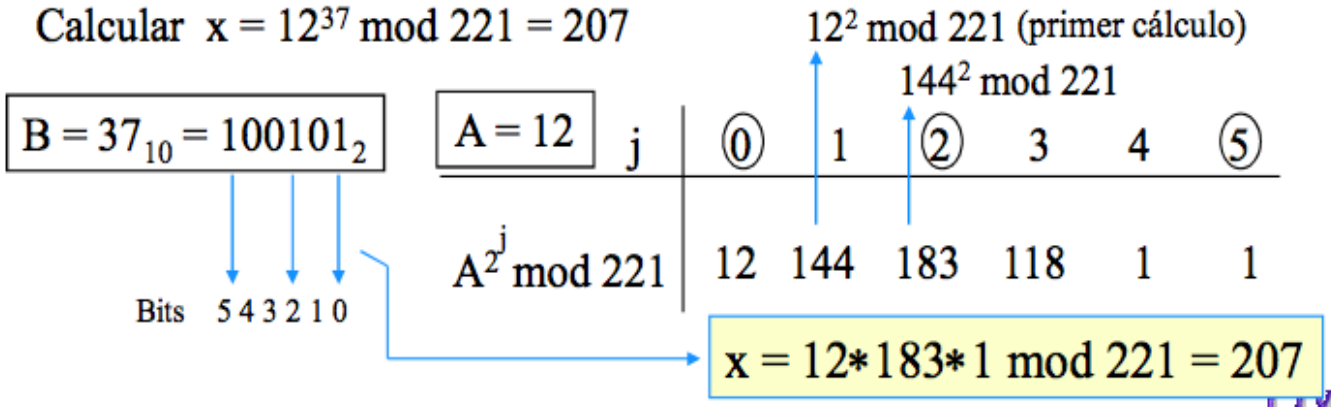
Ataque Person-in-the-Middle a Diffie-Hellman



Solución: Certificados con Firma Digital

Algoritmo de Exponenciación Rápida

- Calcular $a^b \pmod m$
- Representación binaria de b: $b = 2^0b_0 + 2^1b_1 + \ldots + 2^{n-1}b_{n-1}$
- Se calculan los productos a^{2^j} con j = 0 hasta n-1, siendo n el número de bits que representan el valor b en binario.
- Sólo se toman en cuenta los productos en los que en la posición j del valor b en binario aparece un 1.



Exponenciación Rápida

- El algoritmo usa un bucle en el que en cada iteración se generan y multiplican los cuadrados correspondientes, al mismo tiempo que se va dividiendo el exponente **b** entre 2

Entrada: números enteros a (base), b (exponente), m (módulo)

Salida: un entero $x = a^b \bmod m$

```
función Exponenciación (entero a, entero b, entero m) : entero
    x:=1
    y:= a mod m
    while (b >0) and (y>1) do
      begin
          if (b es impar) then
                    x:= (x*y) mod m
                    b:=b-1
          else
                    y:= (y*y) mod m
                    b:= b/2
          endif
      endwhile
    return (x)
end función
```



Exponenciación rápida

- $a^b \pmod m = 2^{28} \pmod{79} = 2^4\ 2^8\ 2^{16} = 16*19*45 = 67*45 = 13$

y	b	x
2	28	1
$2^2=4$	14	
$4^{2=}16$	7	
	6	16
$16^{2=}256\cong19$	3	
	2	$16*19=304\cong67$
$19^{2=}361\cong45$	1	
	0	$67*45=3015\cong13$

Seguridad del Algoritmo de Diffie-Hellman

- Inventado en los 70, fue la primera forma de acordar una clave secreta a través de un canal público.
- Se basa en la dificultad de resolver el **logaritmo discreto**, así que un ordenador cuántico suficientemente potente (usando el algoritmo de Shor) permite romper Diffie-Hellman casi instantáneamente.
- Por eso, en 2026, se está retirando el DH tradicional.

La Amenaza Cuántica: Algoritmo de Shor

- Algoritmo cuántico desarrollado por Peter Shor en **1994** que permite **factorizar** números enteros de manera extremadamente eficiente.
- Su relevancia es crítica porque tiene el potencial de romper la mayoría de los sistemas de criptografía asimétrica actuales, como **RSA**, que basan su seguridad en la dificultad técnica de factorizar números muy grandes.
- Además, permite romper esquemas de establecimiento de clave secreta compartida como el de **Diffie-Hellman**, basado en el problema del **logaritmo discreto**.
- El proceso se divide en 3 fases:
  - Fase Clásica: Prepara el problema reduciendo la **factorización a la búsqueda del periodo** de una función matemática específica.
  - Fase Cuántica: Utiliza la **Transformada Cuántica de Fourier para encontrar el periodo** de forma paralela.
  - Se vuelve a la fase clásica para **calcular a partir del periodo, los factores primos mediante el Máximo Común Divisor**.
- A pesar de su potencia teórica, su implementación práctica a gran escala sigue siendo un reto por dos motivos principales:
  - **Requerimiento de Cúbits**: Para romper el cifrado RSA actual, se necesitarían millones de cúbits estables, mientras que los procesadores más avanzados actuales, como el IBM Condor, apenas superan los 1,000.
  - **Errores**: La construcción de puertas lógicas es extremadamente compleja y propensa a errores en el hardware actual.

Computación Cuántica

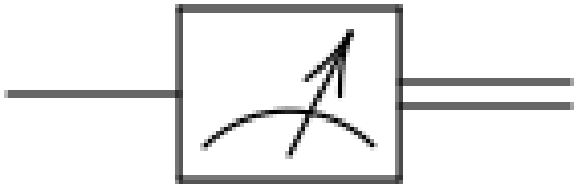
- **Un cúbit puede estar en uno de los dos estados básicos $|0\rangle$ y $|1\rangle$, pero también puede encontrarse en un estado de superposición combinación de esos dos estados:**

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

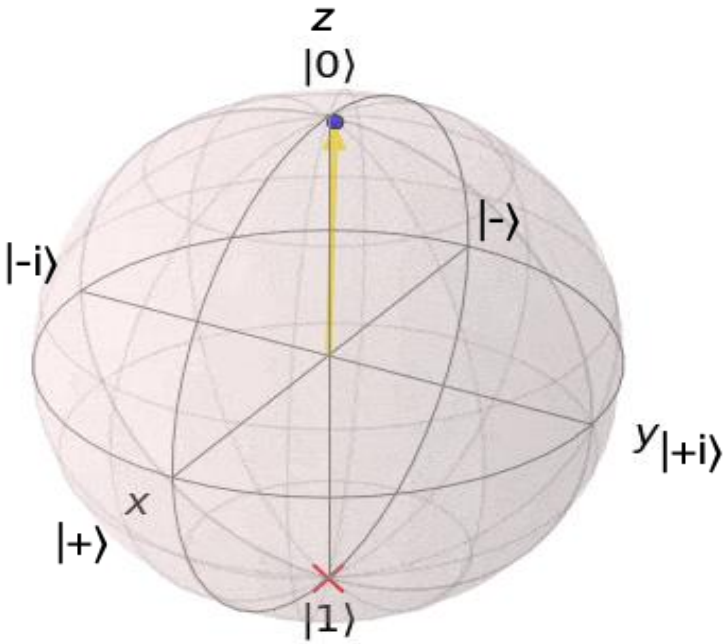
$$\alpha, \beta \in \mathbb{C}$$

$$\alpha^2 + \beta^2 = 1$$

- Mientras un bit clásico puede tomar solo los valores 0 o 1, un cúbit contiene ambos valores ($|0\rangle$ y $|1\rangle$), y solo fija uno de ellos cuando es **medido**.



Cúbit

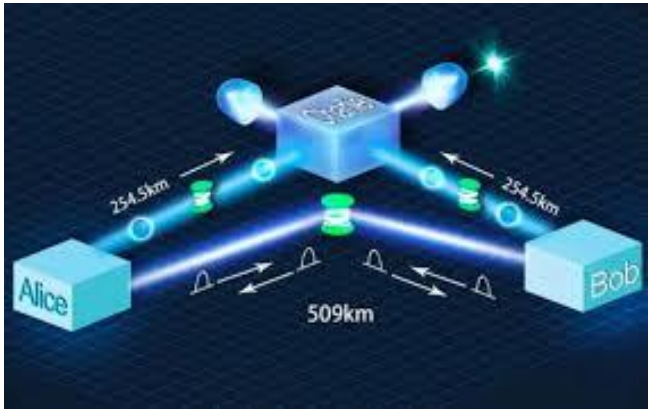


$$|0\rangle = \frac{1}{\sqrt{2}}(|+\rangle + |-\rangle)$$
$$|1\rangle = \frac{1}{\sqrt{2}}(|+\rangle - |-\rangle)$$

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$
$$|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

La Solución Cuántica: Quantum Key Distribution

- **Distribución Cuántica de Claves o Quantum Key Distribution (QKD): Protocolo criptográfico basado en propiedades de la mecánica cuántica, que permite que dos partes compartan una clave secreta aleatoria usando solo un canal público.**



Protocolo BB84





Quantum transmission & detection	ALICE sends photons	↑	↓	→	←	→	→	↓	↓	←	←
	ALICE's random bits	0	1	0	1	0	0	1	1	1	1
	BOB's detection events	←	←	↓	↑	→	→	→	↓	←	↓
	BOB's detected bit values	1	1	1	0	0	0	0	1	1	1
Public discussion (i.e., sifting)	BOB tells ALICE the basis choices he made	←	←	↓	↑	→	→	→	↓	←	↓
	ALICE tells BOB which bits to keep					✅	✅		✅	✅	
	ALICE and BOB's shared sifted key	-	-	-	-	0	0	-	1	1	

El Sucesor Post-Cuántico a la Distribución de Claves

- Los Mecanismos de Encapsulación de Claves o Key Encapsulation Mechanism (KEM) son la evolución moderna y "resistente a la cuántica" del intercambio de claves.
- Mientras que en DH ambas partes "construyen" la clave juntas, en un KEM una parte genera una clave simétrica y la encapsula usando la clave pública del receptor.
- En 2024 el NIST aprobó el estándar ML-KEM, que hoy usa Google Chrome o Cloudflare, basado en retículos matemáticos.

Key Encapsulation Mechanism (KEM)

